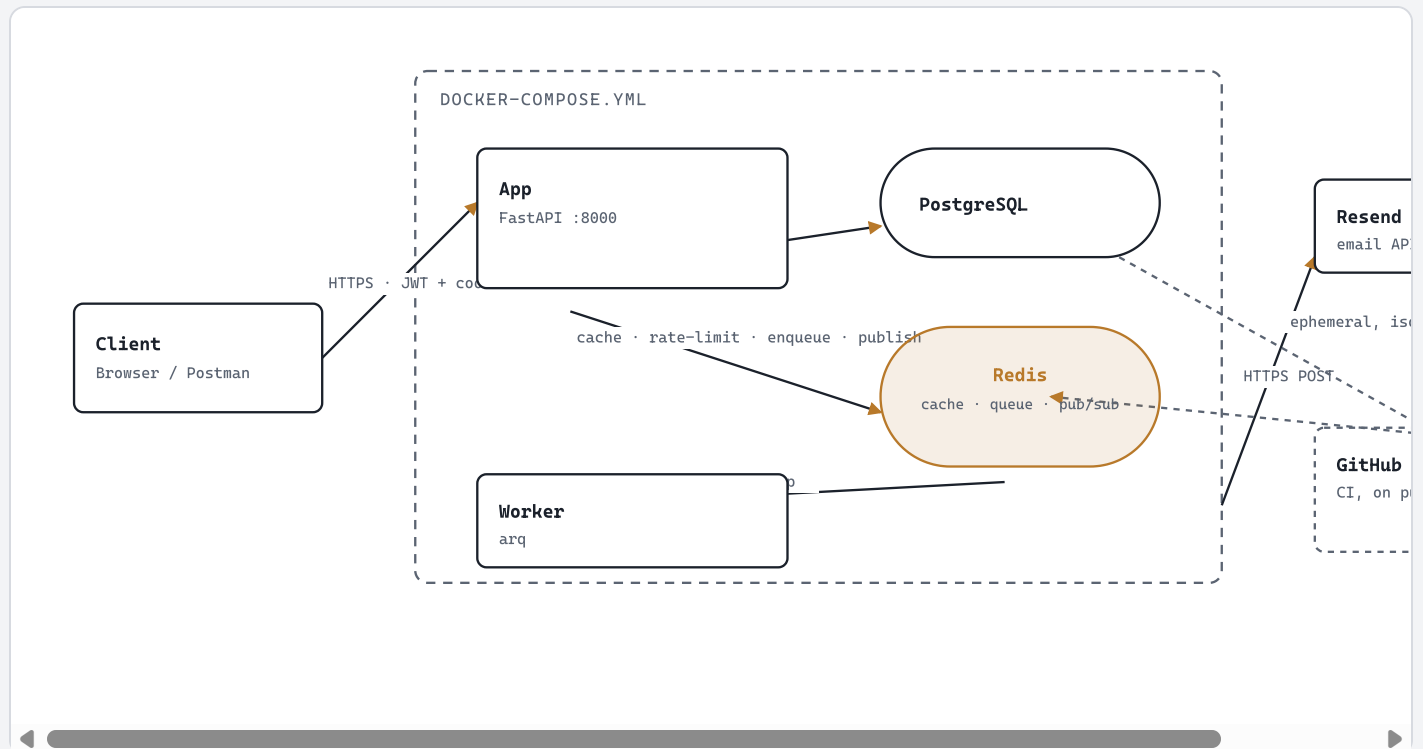


Every concept, one page – diagram, refresher, real code

32 concepts covered while building this project, each with the same shape: a diagram of the mechanism, a short refresher of the idea and why it exists, the technical details worth remembering, and the actual code from this repo — not generic boilerplate.

SYSTEM ARCHITECTURE



One Redis instance, three roles (job queue, cache, pub/sub), separated only by key-naming convention. App and Worker share one built image, differing only in startup command.

DATABASE SCHEMA

USERS

id PK
username
email
hashed_password
is_active
created_at

USER_TASKS

id PK
user_id FK
title
description
is_completed
created_at

REFRESH_TOKENS

id PK
user_id FK
jti
expires_at
revoked
created_at

ACTIVITY_LOGS

id PK
user_id FK
action
detail
created_at

ROLES

id PK
name

PERMISSIONS

id PK
name

- **USERS** 1 → * **USER_TASKS** · 1 → * **REFRESH_TOKENS** · 1 → * **ACTIVITY_LOGS**
(onDelete="CASCADE")
- **USERS** * ↔ * **ROLES** via **user_roles** (association table)
- **ROLES** * ↔ * **PERMISSIONS** via **role_permissions** (association table)

Every FK cascades on delete (onDelete="CASCADE"). **user_roles** / **role_permissions** are plain association tables carrying the two many-to-many relationships, not full ORM models.

PHASE A – FOUNDATIONS

01 Configuration Management

FOUNDATIONS

.env file

real env vars

used everywhere via import



BaseSettings



validated once,
at import

settings singleton



The idea: `os.getenv` returns raw strings (or `None`), fails late (crashes on first use, not at boot), and scatters config reads across the codebase. Pydantic's `BaseSettings` validates every value **at startup** — missing or malformed config refuses to boot, with a clear error, instead of surfacing at 3am in production.

DETAILS

- Priority: real env var > `.env` file > class default — dev/staging/prod need zero code changes.
- `settings = Settings()` constructed **once**, at module import — a validated singleton, not re-parsed per call.
- Type coercion is automatic: `"False"` in `.env` becomes a real Python `bool`, not a truthy string.

CODE — SRC/UTILS/SETTINGS.PY

```
# fail fast: app refuses to boot if a required field is missing
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    DB_CONNECTION: str
    JWT_SECRET_KEY: str
    JWT_ACCESS_EXPIRE_MINUTES: int = 30
    REDIS_URL: str
    COOKIE_SECURE: bool = False    # False in dev, True behind real HTTPS

settings = Settings()    # singleton — import this, never re-instantiate
```

02 SQLAlchemy Core Objects

FOUNDATIONS

engine

HOW to talk to the DB — connection pool, dialect



bound to

sessionmaker

HOW to stamp fresh Sessions



produces

Session (per request)

Unit of Work

Base

class-to-table registry

Three objects, three separate questions. **engine**: how do I physically talk to the DB (pools connections — a taxi company, not a car built per ride). **Base**: how do my Python classes map to tables (a registry — every subclass "enrolls"). **sessionmaker**: how do I manage one transaction (a Session is a shopping cart tracking adds/changes, all-or-nothing at `commit()`).

DETAILS

- `create_engine()` is lazy — connects to nothing until the first real query.
- Long-lived infra (engine/Base/factory) created **once per app**; Sessions are **one per request**, then closed.
- Two requests sharing a Session = two strangers sharing one shopping cart.

CODE — SRC/UTILS/DB.PY

```
engine = create_async_engine(url=settings.DB_CONNECTION.replace("postgresql://",
LocalSession = async_sessionmaker(bind=engine, expire_on_commit=False)

class Base(DeclarativeBase):
    pass

async def get_db():
    session = LocalSession()
    try:
        yield session          # request handling happens here
    except Exception:
        await session.rollback()
        raise
    finally:
        await session.close() # same setup→yield→cleanup shape as middleware/li
```

03 Declarative Models (2.0 style)

FOUNDATIONS

`Mapped[int]`
Python-side truth

→
infers
column type

`mapped_column()`
DB-only extras

→

real table column

The old `Column` style was writing DDL in Python clothes — no autocomplete, no type checking. 2.0 style: write an ordinary typed class, annotate DB-only details on the side. `Mapped[str | None]` reads exactly like an Optional field in a Pydantic DTO — nullability lives in the type itself.

DETAILS

- `Mapped[X]` = what the attribute is in Python; SQLAlchemy infers the SQL type from it.
- `mapped_column(...)` = extras only (`primary_key` , `ForeignKey` , `unique`) — omit entirely if there are none.
- Querying: `select(Task).where(...)` via `session.execute` — `session.query()` is the legacy 1.x idiom.

CODE — SRC/TASK/MODELS.PY

```
class Task(Base):
    __tablename__ = "user_tasks"
    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id", ondelete="CASCADE"))
    title: Mapped[str] = mapped_column(String(100))
    description: Mapped[str | None]          # nullable - the | None IS the schema
    is_completed: Mapped[bool] = mapped_column(default=False)
    owner: Mapped["User"] = relationship(back_populates="tasks")
```

04 Feature Routers

FOUNDATIONS

task/router.py
prefix /tasks

user/router.py
prefix /users

admin/router.py
prefix /admin

→
include_router main.py — app

Mental model — a shopping mall: app is the building; each `APIRouter` is a store organizing its own shelves without knowing its floor. `include_router` is the mall

assigning the unit number, prepending the **prefix**. Routes inside a router are written relative — `@router.get("/{id}")`, not `"/tasks/{id}"`.

DETAILS

- Solves file organization *and* dependency direction — features never import `main.py` ; `main.py` assembles features.
- `from main import app` inside a feature file = circular import, the exact design this pattern prevents.
- Forgetting `include_router` → the router silently doesn't exist. Same "registered nowhere" bug family as a forgotten migration import.

CODE — MAIN.PY

```
from src.task.router import router as task_router
from src.admin.router import router as admin_router
# ...
app.include_router(task_router)
app.include_router(admin_router) # → every /admin/* route now exists
```

05 Response Models & Serialization

FOUNDATIONS



Pydantic's default read protocol is **keys** (`data["title"]`) — its home turf is the JSON boundary. ORM objects store data as **attributes** (`task.title`).

`from_attributes=True` grants a second protocol: "not a dict? try `getattr` on each field instead."

DETAILS

- Modern FastAPI's `response_model` path passes `from_attributes=True` per-call automatically — verified by reading `fastapi/_compat/v2.py`, not assumed from older tutorials.

- Still required for **manual** conversions — `TaskResponseSchema.model_validate(task)` in controllers, tests, Pub/Sub payloads.
- Put it on schemas whose purpose is "built from ORM objects" — leave it off plain request DTOs.

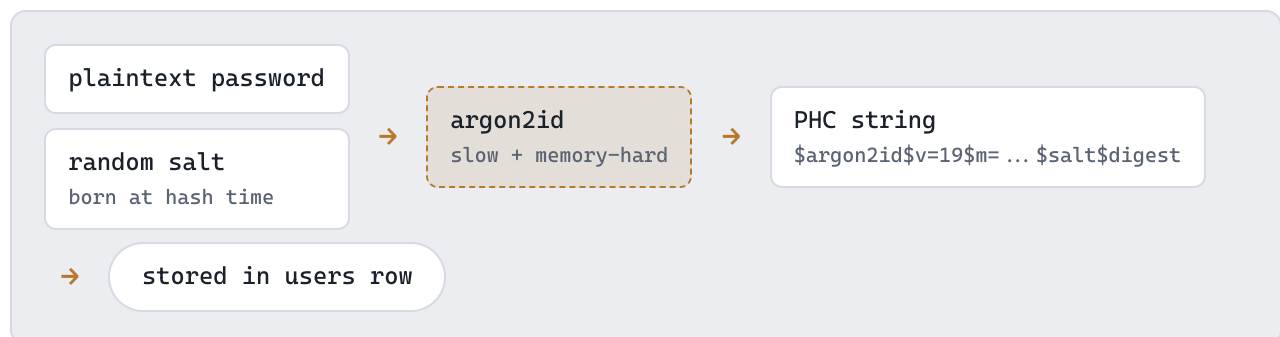
CODE — SRC/TASK/DTOS.PY

```
class TaskResponseSchema(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    title: str
    is_completed: bool
    created_at: datetime
```

PHASE B — AUTH CORE

06 Password Hashing

AUTH CORE



Threat model: assume the DB eventually leaks. The question isn't "prevent the breach," it's "what does the attacker get." Hashing is one-way — meat grinder analogy: mince never becomes steak again, but two batches from the same cut match. Argon2id over bcrypt/sha256 specifically because it's **memory-hard** — GPU farms can't feed thousands of parallel cores enough RAM per guess.

DETAILS

- Salt's job is **uniqueness**, not secrecy — kills rainbow tables, lives inside the PHC string itself, right in the row.
- Verify uses **constant-time comparison** — naive `==` leaks timing side-channels.

- CPU-bound → wrapped in `run_in_threadpool` at the call site, never awaited eagerly (a real bug caught by the test suite — see 20).

CODE — SRC/UTILS/SECURITY.PY

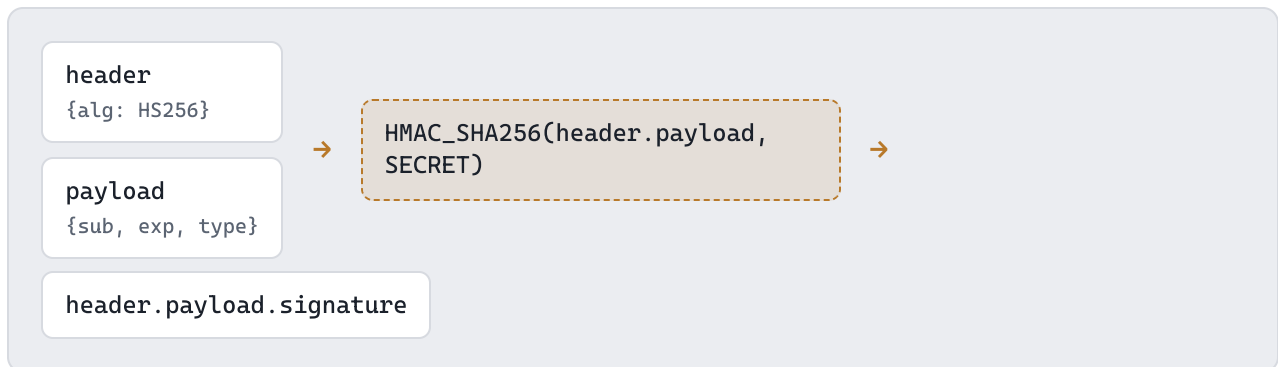
```
password_hash = PasswordHash.recommended() # policy, not hand-picked crypto pa

def hash_password(password: str) → str:
    return password_hash.hash(password)

def verify_password(password: str, hashed: str) → bool:
    return password_hash.verify(password, hashed) # constant-time internally
```

07 JWT Access Tokens

AUTH CORE



Festival wristband, not a guest list: the server writes a signed note once, the client carries it, the server verifies its own signature instead of a DB lookup every request. The payload is **readable by anyone** (base64, not encryption) — the signature makes it tamper-proof, not secret. Claims carry identifiers, never secrets.

DETAILS

- `sub` must be a **string** (RFC 7519) — proven live: pyjwt mints `sub: 2` fine but rejects it on decode.
- Must use `datetime.now(timezone.utc)` — a naive local datetime silently makes tokens live hours too long.
- `options={"require": ["exp", "sub"]}` — without it, a token with no `exp` is immortal.

CODE — SRC/UTILS/SECURITY.PY


```
def create_access_token(user_id: int) → str:
    payload = {
        "sub": str(user_id), # str at the bound
        "exp": datetime.now(timezone.utc) + timedelta(minutes=settings.JWT_ACCESS_EXPIRE_MINUTES),
        "type": "access",
    }
    return jwt.encode(payload, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)
```

08 Bearer Auth & the Dependency Chain

AUTH CORE

Client **bearer_scheme** **jwt.decode** **db.get(User)**

Client → **bearer_scheme** Authorization: Bearer eyJ...

bearer_scheme → **Client** 401 if missing / malformed header

bearer_scheme → **jwt.decode** raw token

jwt.decode → **Client** 401 if tampered / expired

jwt.decode → **db.get(User)** sub → user_id

db.get(User) → **Client** 401 gone / 403 inactive

db.get(User) → **Client** verified User → endpoint runs

Three guards, each strictly dumber than the next, each unreachable unless the previous passed. **HTTPBearer** knows only HTTP headers (a bouncer's reflex). **jwt.decode** knows crypto and claims. The final DB check knows application state. The endpoint inherits all three guarantees as one typed parameter.

DETAILS

- **401** = identity claim failed (any reason, including "user no longer exists" — don't leak which).
- **403** = identity established, access denied (inactive).
- **Depends()** resolves twice in time: once at startup (builds the graph, marks OpenAPI security), once per-request (leaves-first, cached — **get_db** appearing twice in the tree still runs once).

CODE — SRC/UTILS/DEPENDENCIES.PY

```

async def get_current_user(credentials: BearerCredentials, db: DbSession):
    payload = decode_token(credentials.credentials)
    if payload.get("type") != "access":
        raise HTTPException(401, "Invalid token type") # refresh token != bearer
    user = await db.get(User, int(payload["sub"]))
    if not user: raise HTTPException(401)
    if not user.is_active: raise HTTPException(403, "User account is inactive")
    return user

CurrentUser = Annotated[User, Depends(get_current_user)]

```

09 Refresh Token Rotation + Reuse Detection

AUTH CORE

Client **Server**

Client → **Server** POST /refresh (old token)

Server ↻ **Server** mark old token revoked = True

Server → **Client** NEW refresh + access token

attacker replays the OLD token later

Client → **Server** POST /refresh (old, already-revoked token)

Server → **Client** 401 + REVOKE ALL sessions for this user

Every refresh mints a brand-new token and revokes the one just used. A revoked token being presented again is a strong signal of theft — not "expired," *stolen and now in two hands* — so the response isn't just "deny," it's "kill every session this user has."

DETAILS

- `jti` (unique, not secret) identifies the row; a separate CSPRNG-backed value would be needed if the token itself had to be unguessable server-side.
- Real bugs caught building this: `db.add(RefreshSchema( ... ))` instead of the ORM model; a missing `payload["type"] == "access"` check let refresh tokens work as permanent bearer creds.

CODE — SRC/USER/CONTROLLER.PY

```

if token_row.revoked:
    # reuse of an already-rotated token - treat as compromise
    await db.execute(update(RefreshToken).where(RefreshToken.user_id == token_row.user_id).values(revoked=True))
    await db.commit()
    raise HTTPException(401, "Refresh token reuse detected - all sessions revoked")

token_row.revoked = True # rotate: this one is now spent
await db.commit()
return await issue_tokens(user, db)

```

10 Ownership Authorization (404 Privacy)

AUTH CORE

GET /tasks/7 (user B)



task exists AND task.user_id == B ?

no row at all →

404

exists, but belongs to A →

404 - SAME response

exists, belongs to B →

200 + task

"Not there" and "not yours" are made indistinguishable on purpose — a real 404 and a privacy-denial 404 look identical to the caller, so probing task IDs can't be used to confirm which ones exist under someone else's account.

DETAILS

- Proven with a live two-user adversarial test — identical 404s for "doesn't exist" and "exists but isn't yours."
- The same pattern generalizes: any resource with an owner column gets this check before any other logic runs.

CODE - SRC/TASK/CONTROLLER.PY

```

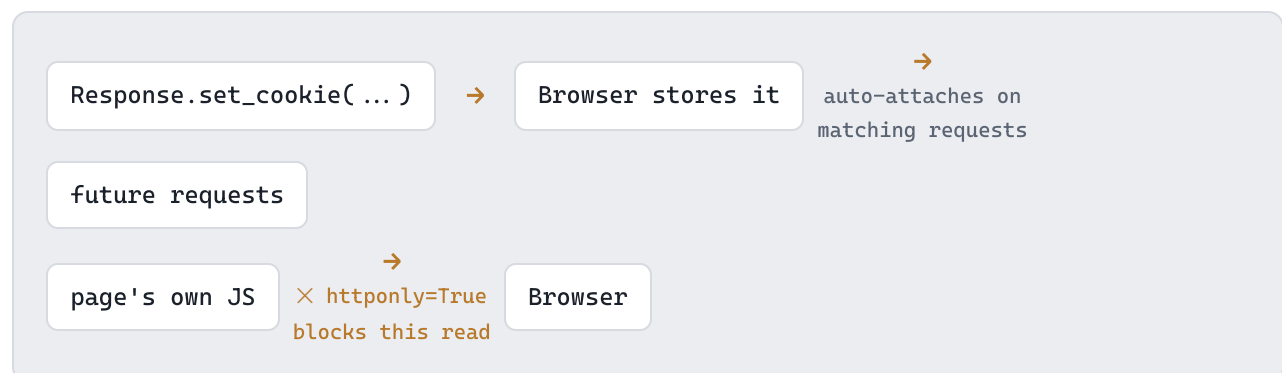
async def get_task_or_404(db: AsyncSession, task_id: int, user_id: int):
    task = await db.get(Task, task_id)
    if not task or task.user_id != user_id:
        raise HTTPException(404, f"Task {task_id} not found") # same 404 either
    return task

```

PHASE C – WEB SECURITY

11 httpOnly Cookies

WEB SECURITY



A cookie is data the server asks the browser to store and **reattach automatically** — nobody re-sends it manually, which is both the point and the risk. The refresh token (a 7-day credential) moved here specifically because `httponly=True` blocks any JS — yours, a library's, an XSS payload — from reading its value at all. The access token can't get this treatment: client JS must read it to build the **Authorization** header, so it's contained by short expiry instead.

DETAILS

- Four attributes, one job each: `httponly` (blocks JS reads), `secure` (HTTPS-only transport), `samesite` (the CSRF-relevant knob), `path` (scopes which URLs get it).
- Real bug caught live: a cookie path scoped to `/users/refresh` silently never reached the sibling route `/users/logout` — fixed by picking the narrowest common-ancestor path, `/users`.

CODE – SRC/USER/ROUTER.PY

```
response.set_cookie(value=token_payload["refresh_token"], key="refresh_token",
                    httponly=True, secure=settings.COOKIE_SECURE, samesite="lax",
                    path="/users", max_age=settings.JWT_REFRESH_EXPIRE_DAYS*24*60*60)
```

12 CSRF – Double-Submit Cookie

WEB SECURITY

Browser (auto) Page JS (manual) Server

Two INDEPENDENT channels must agree

Browser → Server Cookie: csrf_token=abc (auto-attached)

Page JS → Server X-CSRF-Token: abc (read via document.cookie, sent manually)

Server ⇄ Server cookie value == header value?

Server → Page JS match → proceed / mismatch → 403

The server generates a random token at login and then **forgets it** — no DB row, no session store. The whole check is one line: does the incoming cookie equal the incoming header, right now. It works because a forged cross-site request can trigger the automatic channel (the cookie) but can't replicate the manual one — evil.com's JS calling `document.cookie` for a domain it isn't running on gets nothing, by the browser's own same-origin rule.

DETAILS

- `secrets.token_urlsafe(32)`, not `uuid4()` — this token's job is being *unguessable*, not merely unique.
- `secrets.compare_digest(None, None)` raises `TypeError`, doesn't return `False` — the most common real request (no CSRF header yet) would 500 without a short-circuit guard.
- `dependencies=[Depends(verify_csrf)]` — a **bare** function in that list crashes the whole app at import time, not just the route.

CODE – SRC/UTILS/DEPENDENCIES.PY

```
def verify_csrf(csrf_token: Annotated[str|None, Cookie()]) = None,
                X_CSRF-Token: Annotated[str|None, Header()] = None):
```

```
if not csrf_token or not X_CSRF-Token or not secrets.compare_digest(csrf_token, X_CSRF-Token):
    raise HTTPException(403, "CSRF token missing or invalid") # short-circuit
```

13 CORS

WEB SECURITY

Browser :5500 **API** :8000

Browser → **API** OPTIONS /users/login (preflight)

API → **Browser** Access-Control-Allow-Origin: :5500

only now does the REAL request go out

Browser → **API** POST /users/login

API → **Browser** 401 (JS on the page CAN read this)

A **browser** story, not a server story — curl/Postman/TestClient never see CORS at all. Same-Origin Policy blocks cross-origin JS from *reading a response* by default; CORS headers are the server opting **out** of part of that lockout for named origins. Not auth — it decides which pages' JS may read responses, not who may access data.

DETAILS

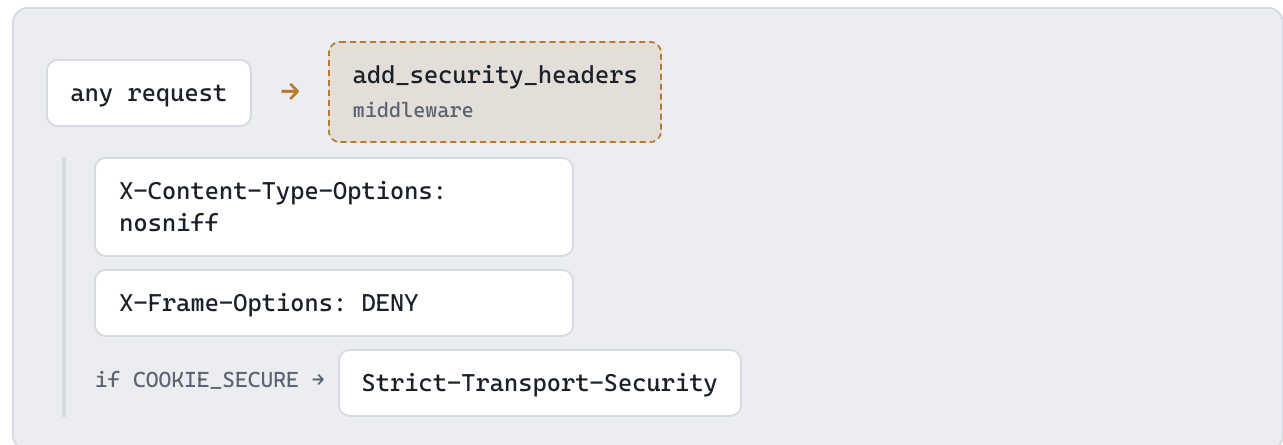
- `allow_origins=["*"]` + `allow_credentials=True` together is rejected outright by browsers — genuinely not an option once real cookies are in play.
- Multi-middleware ordering rule, verified both directions: **last registered = outermost**, first to see the request, last to touch the response.

CODE — MAIN.PY

```
app.add_middleware(CORSMiddleware,
    allow_origins=["http://localhost:5500"], allow_credentials=True,
    allow_methods=["*"], allow_headers=["*"])
```

14 Security Headers

WEB SECURITY



Browser-enforced instructions, stamped on **every** response — textbook "for every request" middleware. `nosniff` stops a browser guessing a different content type than declared. `X-Frame-Options: DENY` closes off clickjacking (a real target here: `/docs'` own "Execute" button, invisibly framed). HSTS closes the SSL-stripping window on a user's very first plaintext request.

DETAILS

- CSP deliberately skipped — this app only returns JSON, no script-execution surface for a content policy to restrict.
- HSTS gated behind `settings.COOKIE_SECURE` — sending it on plain `localhost` dev could make a browser refuse to connect at all for the whole `max-age` window.

CODE – SRC/UTILS/SECURITY_HEADERS.PY

```
async def add_security_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    if settings.COOKIE_SECURE:
        response.headers["Strict-Transport-Security"] = "max-age=63072000; includeSubDomains"
    return response
```

15 HTTPS / TLS

WEB SECURITY

Browser Caddy / LB (holds cert) FastAPI app

Browser → Proxy TLS handshake, verifies cert against trusted CA

Browser → Proxy encrypted request

Proxy → App PLAIN http, internal network only

App → Proxy plain response

Proxy → Browser re-encrypted response

HTTPS = HTTP wrapped in an encrypted tunnel — app code never changes. Asymmetric crypto bootstraps a shared symmetric session key once (slow, used briefly); everything after uses fast symmetric encryption. A **certificate** is a passport analogy: a CA signs "this domain owns this public key" with its own private key; the browser verifies that signature locally against a CA it already trusts. TLS **terminates at a reverse proxy**, not inside the FastAPI process — the internal hop stays plain HTTP because it never leaves a trusted network boundary.

DETAILS

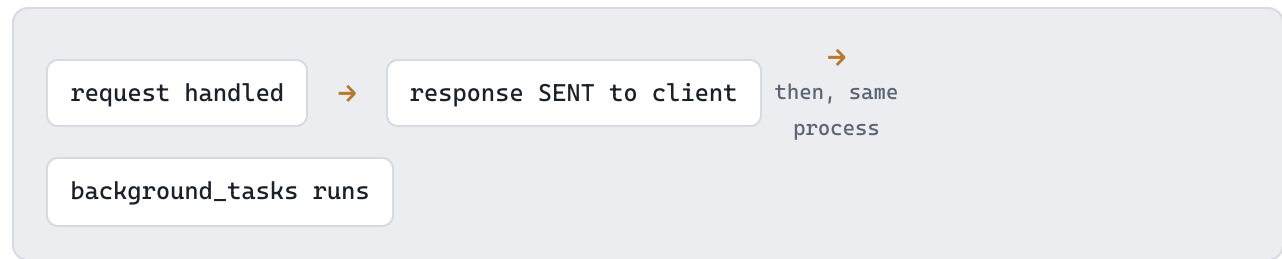
- **ACME** (Let's Encrypt) proves domain control via a challenge — e.g. serve a specific token at `/.well-known/acme-challenge/<token>` — then signs automatically, no human review.
- `localhost` is structurally unreachable from outside — real deployment needs an actually different, internet-routable machine.

THE ONE APP-CODE CHANGE THIS TOPIC PRODUCES

```
secure=settings.COOKIE_SECURE   # False in dev, True once genuinely behind HTTPS
```


16 BackgroundTasks

BACKGROUND WORK



The waiter analogy: bring the food first, update the inventory sheet after. The criterion: *background what the response merely triggers; foreground what it asserts*. Email on register — the 201 means "account exists," true at commit, the email arriving later changes nothing. Hashing — never; the 201 semantically asserts "credential stored."

DETAILS

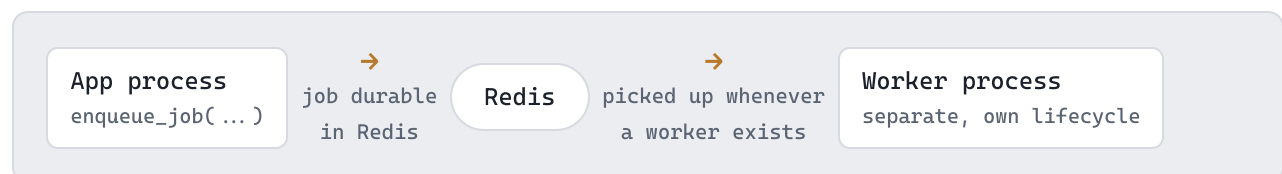
- Dies with the process — no retries, no persistence. A restart mid-send = the task silently never existed. Loss-tolerable work only.
- Anything that outlives the request must open its **own** DB session — the request's `get_db` session isn't guaranteed to still be usable.

CODE — SRC/TASK/ROUTER.PY

```
background_tasks.add_task(log_activity, current_user.id, "task_created", f"task  
return new_task # response goes out; log_activity runs after, in its own sessi
```

17 Redis Task Queue (arq)

BACKGROUND WORK



Extends BackgroundTasks' own criterion: *must-happen, not loss-tolerable* → *a notebook that survives restarts*. A queued job sat in Redis ~9 real minutes, untouched, until a worker

started — and was still processed correctly. That's the entire value proposition, proven live, not asserted.

DETAILS

- Chosen over Celery: async-native, Redis-only, no new infrastructure concept beyond what the app already uses.
- `WorkerSettings.functions = [ ... ]` is the same "registered nowhere = doesn't exist" family as `include_router`.
- Default retry behavior (traced from source, not assumed): `max_tries=5`, `retry_jobs=True` — but only works if the task actually `raise`s on failure.

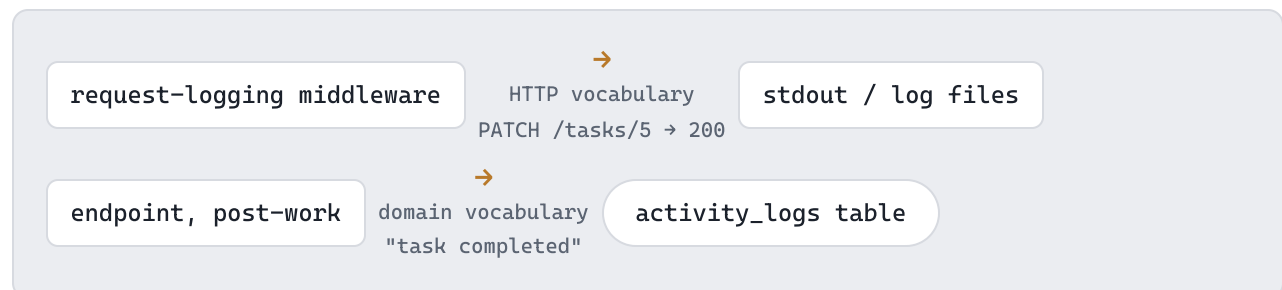
CODE — SRC/UTILS/ARQ_FUNCTIONS.PY

```
async def send_welcome_email(ctx, email: str):    # ctx always first, arq's conveyer
    async with httpx.AsyncClient() as client:
        response = await client.post("https://api.resend.com/emails", ...)
        response.raise_for_status()    # without this, a failed send looks "successful"

class WorkerSettings:
    functions = [send_welcome_email]
    redis_settings = REDIS_SETTINGS
```

18 Activity Feed (Domain Events)

BACKGROUND WORK



Two different kinds of "logging," decided by audience. Middleware records what happened to the *request* — HTTP facts, for operators, zero endpoint knowledge needed. The activity feed records what happened to the *domain* — "completed a task," for the user, a product feature — knowledge middleware structurally can't have, since from `PATCH /tasks/5 → 200` alone it can't tell completed-vs-renamed.

DETAILS

- Sat write-only for most of the project's life — `GET /activity` to read it back came much later, closing a real, previously-flagged gap.

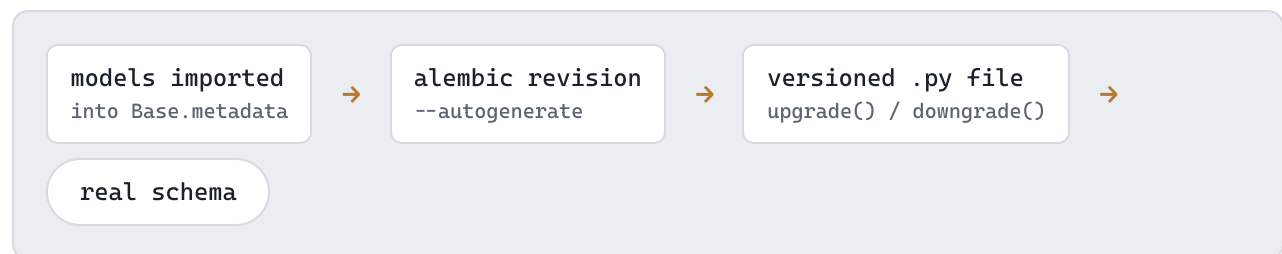
CODE — SRC/UTILS/ACTIVITY.PY

```
async def log_activity(user_id: int, action: str, detail: str | None = None):
    async with LocalSession() as session:    # own session — outlives the request
        session.add(ActivityLog(user_id=user_id, action=action, detail=detail))
    await session.commit()
```

PHASE E — DATA & PERFORMANCE

19 Alembic Migrations

DATA & PERFORMANCE



`create_all` only creates *missing* tables, never alters existing ones — real apps need git-commit-style, versioned, reversible schema history. Autogenerate only sees models that are actually imported into the process running it — `migrations/env.py` has to import every model module explicitly, or new tables go undetected silently, no error.

DETAILS

- Migrations should build seed data with lightweight `sa.table()` / `sa.column()` stand-ins, never the live ORM class — history must stay replayable after models change or get deleted.
- Real drift caught live: two pre- `UTCDateTime` migrations stored naive datetimes as `timestamp without time zone` on Postgres, invisible the whole time on SQLite.

CODE — MIGRATIONS/ENV.PY

```
from src.user.models import User
from src.task.models import Task
from src.authz.models import Role, Permission # must import, or autogenerate s
target_metadata = Base.metadata
```

20 Async SQLAlchemy

DATA & PERFORMANCE

def route
dispatched to worker thread

→
converted

async def route
real await, no thread

→

asyncpg - real non-blocking
I/O

Once one function in the chain is `async`, every caller up the stack has to `await` it — "async is contagious." The reward: one process serves many requests concurrently by yielding the CPU during any real wait, instead of dedicating a thread per in-flight request.

DETAILS

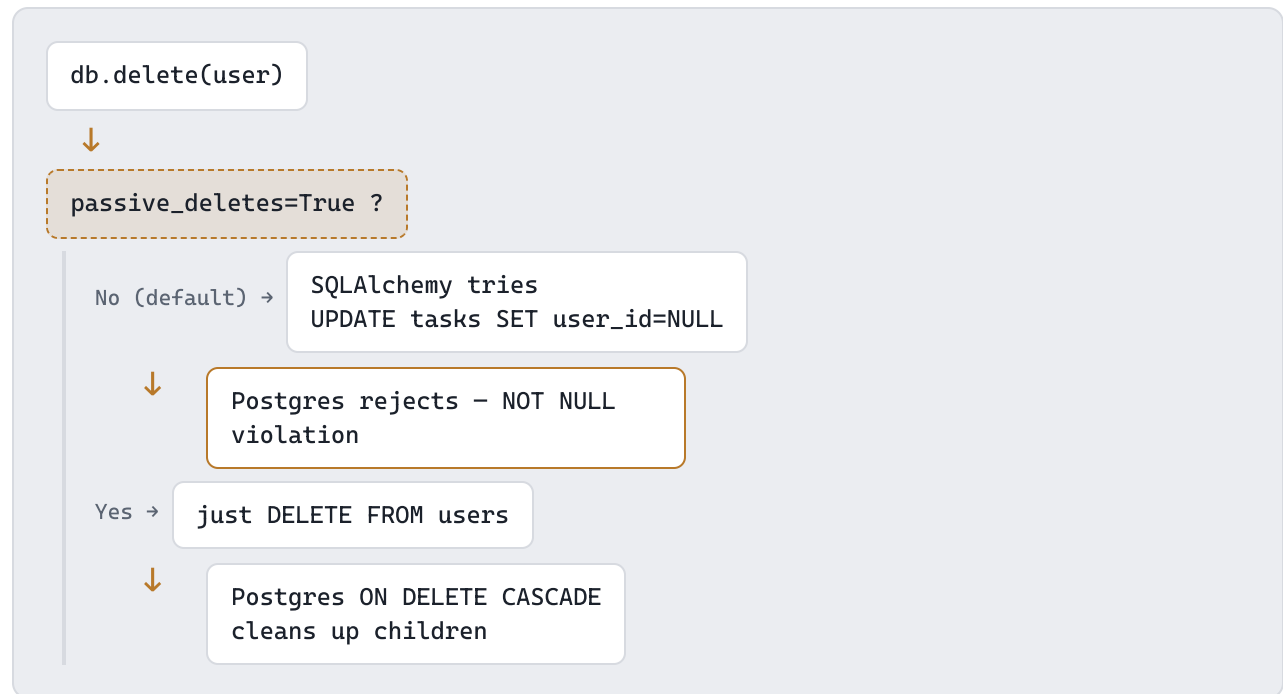
- A genuine authbypass caught by the test suite mid-conversion: a threadpool-wrapped `verify_password` call missing its `await` — an un-awaited coroutine is always truthy, so the password check silently stopped doing anything at all.
- `expire_on_commit=False` is the standard async pattern — without it, reading an attribute right after commit becomes a silent, impossible-in-async lazy reload.

CODE - THE BUG, EXACTLY AS IT SHIPPED AND GOT CAUGHT

```
# before - always True, any password "worked":
if not run_in_threadpool(verify_password, body.password, user.hashed_password):
# after - test_login_wrong_password_returns_401 is what caught the missing await
if not await run_in_threadpool(verify_password, body.password, user.hashed_passw
```

21 Cascade Deletes (passive_deletes)

DATA & PERFORMANCE



Two independent layers. `onDelete="CASCADE"` is a real Postgres constraint — enforced no matter what deletes the row, even raw SQL. `passive_deletes=True` is purely SQLAlchemy-side — it stops the ORM from trying to manage the cascade itself first. Verified live, both directions: without it, deleting a user genuinely **crashes**, not just runs an extra query.

DETAILS

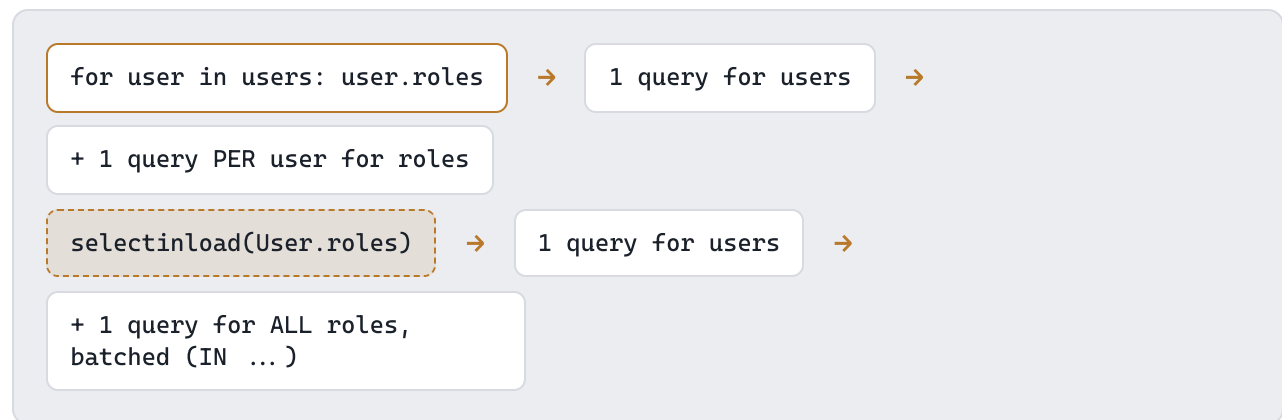
- The failure mode without it: SQLAlchemy defaults to nulling out each child's FK before the parent delete — rejected outright since `user_id` isn't nullable.
- A raw `DELETE FROM users WHERE id=5;` typed into `psql` would cascade correctly with or without this setting — it only governs SQLAlchemy's own Python-side behavior.

CODE – SRC/USER/MODELS.PY

```
tasks: Mapped[list["Task"]] = relationship(back_populates="owner", passive_delet
```

22 N+1 Queries & Eager Loading

DATA & PERFORMANCE



Lazy loading is attribute access that secretly does I/O — fine in sync code, structurally impossible to do silently in async code (SQLAlchemy crashes with `MissingGreenlet` rather than freeze the whole event loop). N+1 is what happens when that lazy fetch is triggered once *per parent, inside a loop*, instead of once, batched, before the loop.

DETAILS

- `selectinload` : 2 queries total, regardless of N — right default for to-many relationships.
- `joinedload` : 1 query via SQL JOIN, but duplicates parent columns once per child on to-many — better suited to to-one.
- Full audit of this app found **zero** active N+1 bugs — every response DTO stays deliberately flat, never serializes a nested relationship.

CODE – THE EXACT LIVE BUG THIS APP HIT

```
# crashed: MissingGreenlet – user already persisted, roles not loaded
user = (await db.scalars(select(User).where(User.email == email))).first()
user.roles.append(admin_role)

# fixed:
user = (await db.scalars(select(User).where(User.email == email)
    .options(selectinload(User.roles)))).first()
user.roles.append(admin_role)
```

23 Redis Cache-Aside

App Redis Postgres

App → Redis GET tasks:{user_id}

ALT – CACHE HIT

Redis → App cached JSON

ELSE – CACHE MISS

App → Postgres SELECT * FROM user_tasks

App → Redis SET tasks:{user_id} ... EX 60

any write → DELETE tasks:{user_id}

Read path: check cache, fall through to DB on miss, write back with a TTL. Write path: invalidate (delete) the key after the DB commit succeeds. Every Redis call wrapped in a resilience guard — a cache outage degrades to "always hit the DB," never fails the request outright.

DETAILS

- Real bug caught live: caching a raw Python list instead of `json.dumps()` -ing it first — broke every cache miss with a `DataError`.
- Deliberately scoped to the list endpoint only, not single-item fetches — those are already a fast PK lookup, caching them doubles the invalidation surface for little gain.

CODE – SRC/TASK/CONTROLLER.PY

```
try:
    cached_value = await redis_pool.get(key)
    if cached_value: tasks = json.loads(cached_value)
except (redis.exceptions.RedisError, json.JSONDecodeError):
    pass # Redis down or malformed – treat exactly like a normal miss
```

24 Pagination & Filtering

fetch/cache FULL list
one key, unchanged



filter in-memory
is_completed / search



slice [start:end]
+ has_next via fetch-one-extra

A per-page cache key was seriously considered and rejected — it reopens the exact problem it solves (which of an unknown set of page-keys needs invalidating on write?). Given a personal task list is realistically small, the whole list stays cached under one key; pagination and filtering are just in-memory views over it, so the 4 existing write-invalidation paths need zero changes.

DETAILS

- `has_next` via fetching `page_size + 1` rows and checking if the extra one showed up — avoids a separate `COUNT` query entirely.
- The response-shape ambiguity this solves: a bare list of exactly `page_size` items can't tell the client "more exists" from "exactly enough to fill this page."

CODE — SRC/TASK/CONTROLLER.PY

```
start = (page - 1) * page_size
end = start + page_size
return {"items": filtered[start:end], "has_next": end < len(filtered)}
```

25 Rate Limiting

DATA & PERFORMANCE

request from IP/email



SET key 0 EX 60 NX



INCR key

> limit → 429

≤ limit → continue

Fixed-window counter — match the tool's weight to the requirement's, no need for token-bucket sophistication just to stop brute-forcing a login form. SET ... NX EX before INCR

closes a real race: a naive **INCR**-then-**EXPIRE** sequence could leave a permanently un-expiring key if the process crashed between the two calls.

DETAILS

- Built as a parameterized factory — one implementation protects `/login` and `/register` with independent counters via different bucket names.
- Test isolation gotcha caught live: `TestClient`'s shared synthetic host meant rate-limit state accumulated *across* unrelated tests until `conftest` flushed Redis per test.

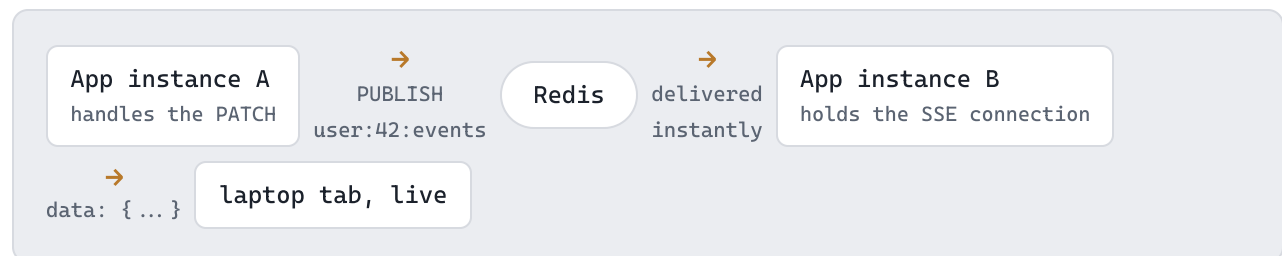
CODE — SRC/UTILS/DEPENDENCIES.PY

```
def rate_limit_ip(bucket: str, ex: int = 60, limit: int = 5):
    async def _rate_limit_ip(request: Request):
        redis_pool: ArqRedis = request.app.state.redis_pool
        key = f"{bucket}:{request.client.host}"
        await check_rate_limit(redis_pool, key, ex, limit)
    return _rate_limit_ip # dependencies=[Depends(rate_limit_ip("register_atte
```

PHASE F — REALTIME & RBAC

26 Server-Sent Events + Redis Pub/Sub

REALTIME



Solves the actual multi-instance problem: an in-memory dict of open connections works at one replica and breaks the instant there's more than one. Pub/Sub is data **in motion** — no key, no persistence; a message with nobody subscribed is simply gone, treated as a hint to refetch, not a guaranteed delivery channel (Postgres stays the source of truth).

DETAILS

- Chose SSE over WebSockets on merit: traffic here is one-directional (mutations go through normal REST), and SSE is plain HTTP with native browser reconnect — no protocol upgrade needed.
- `pubsub.listen()` 's first item is always the subscribe confirmation, not real data — must filter for `type == "message"`.

CODE — SRC/REALTIME/ROUTER.PY

```
async for message in pubsub.listen():
    if message["type"] != "message": continue
    payload = message["data"].decode() # raw bytes — f-string alone gives "b'
    yield f"data: {payload}\n\n"      # SSE wire format — two trailing \n re
```

27 Ticket-Based Realtime Auth

REALTIME

Client Server

Client → Server POST /realtime/ (real Bearer token, normal fetch)

Server ⇄ Server SET sse_ticket:{t} = user_id EX 30 NX

Server → Client {"ticket": t} in JSON body

Client → Server new EventSource("/events?ticket=t")

Server ⇄ Server GETDEL sse_ticket:{t} — atomic, single-use

`EventSource` can't set an `Authorization` header, so the real access token can't ride along — and putting it in a URL means access logs, browser history, `Referer` leaks. The fix: mint a deliberately weak, 30-second, single-use ticket using the real token over a normal `fetch` first — a coat-check tag, not your ID.

DETAILS

- `GETDEL` — atomic read-and-delete in one step, so a ticket can never be redeemed twice even under concurrent requests.
- Short TTL **and** single-use together, not either alone — the ticket is only ever dangerous for the few seconds between minting and redemption.

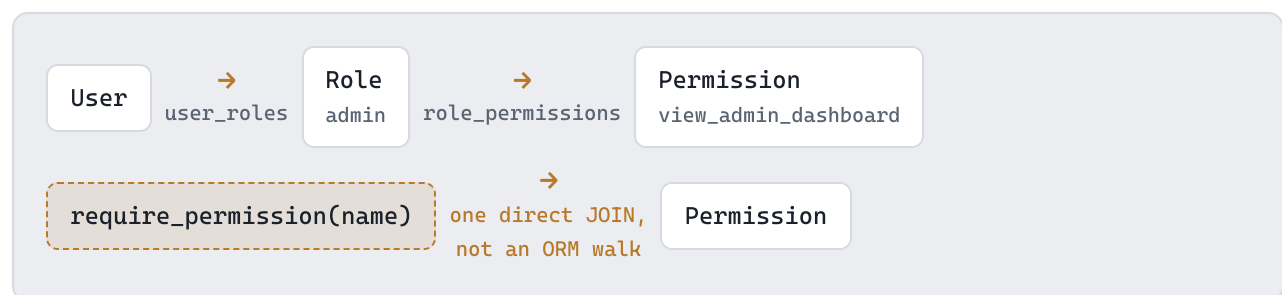
```

user_id = await redis_pool.getdel(f"sse_ticket:{ticket}")
if user_id is None:
    raise HTTPException(401, "Invalid or expired ticket")

```

28 RBAC (Roles & Permissions)

AUTHORIZATION



Routes check **permissions**, never role names directly — the indirection is what lets a new role recombine existing permissions without touching every enforcement point. Built with exactly one real permission (`view_admin_dashboard`), not several pre-invented ones with nothing to enforce yet — the general schema is what future-proofs this, not extra rows nobody checks.

DETAILS

- The permission check is a direct 4-table join, deliberately not `current_user.roles` as an ORM relationship — sidesteps the async lazy-load problem by construction.
- Every user gets `member` automatically at registration; `admin` is never self-assignable through the API.

```

def require_permission(permission_name: str):
    async def _require_permission(current_user: CurrentUser, db: DbSession):
        stmt = (select(Permission.id)
                .join(role_permissions, Permission.id == role_permissions.c.permission_id)
                .join(Role, Role.id == role_permissions.c.role_id)
                .join(user_roles, Role.id == user_roles.c.role_id)
                .where(user_roles.c.user_id == current_user.id, Permission.name == permission_name))
        if (await db.execute(stmt)).first() is None:

```

```
        raise HTTPException(403, "Insufficient permissions")
    return _require_permission
```

PHASE G – INFRASTRUCTURE

29 Middleware

INFRASTRUCTURE

`add_security_headers`
outermost – last registered



`log_requests`



`CORSMiddleware`



`routing → deps → endpoint`

The "for every request" trigger — cross-cutting infrastructure, not a per-route opt-in. `call_next` isn't "the next single step" — calling it runs **everything downstream**, all the way to the endpoint and back, handing back the fully-built response for your own "after" code to still modify before passing it further out.

DETAILS

- Last registered = outermost, first to see the request, last to touch the response — verified both directions, not assumed.
- `contextvars.ContextVar` makes the request-id readable anywhere in the call stack without threading it through every function signature — a plain global would leak across concurrent requests on the event loop.

CODE – SRC/UTILS/LOGGING.PY

```
async def log_requests(request: Request, call_next):
    request_id_var.set(str(uuid.uuid4())) # fresh per request, NOT module-level
    response = await call_next(request)    # runs the entire rest of the app
    logger.info(f"{request.method} {request.url.path} → {response.status_code}")
    return response
```

30 Docker & Docker Compose

INFRASTRUCTURE

one built image
taskmanager-app

app service
cmd: uvicorn

worker service
cmd: arq WorkerSettings

app service depends_on: healthy>

postgres

redis

One image, two services differing only in startup command — same code, no separate build step for the worker. `0.0.0.0` not `127.0.0.1`: the default binds only to loopback inside the container, unreachable through Docker's own port mapping — proven live by running standalone with a stale `localhost`-based config and watching it fail exactly as predicted.

DETAILS

- Layer-cache ordering: `requirements.txt` copied and installed *before* the rest of the source, so the slow step only reruns when dependencies actually change.
- Non-root runtime user — root during build is fine (disposable environment); root handling real traffic is the actual risk.
- `python:3.13-slim` over alpine — native C-extension deps (`asyncpg` , `argon2-cffi`) lack alpine wheels.

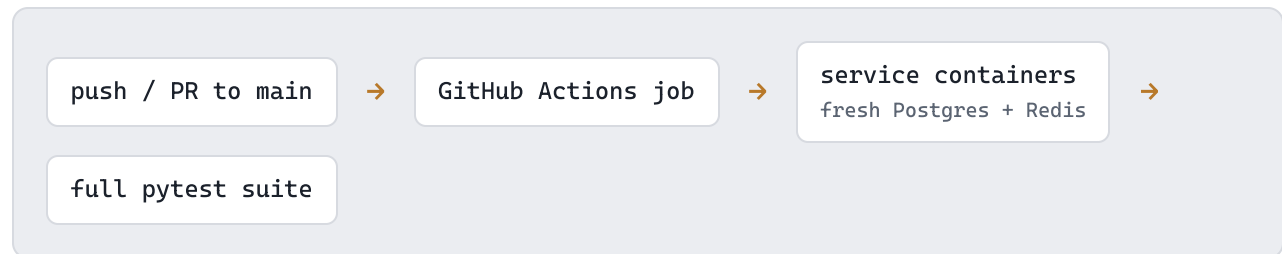
CODE – DOCKERFILE

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # cached unless requirements change
COPY . .
RUN useradd --create-home appuser && chown -R appuser:appuser /app
```

```
USER appuser # root only during the build, never at runtime
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

31 CI (GitHub Actions)

INFRASTRUCTURE



Service containers spin up a genuinely fresh Postgres and Redis per job — same shape as local `docker-compose.yml`, GitHub-orchestrated instead of local. First real run failed for a good first lesson: env vars were scoped only to the test step, not the migration step (which also imports settings) — fixed by moving shared vars to job-level scope.

DETAILS

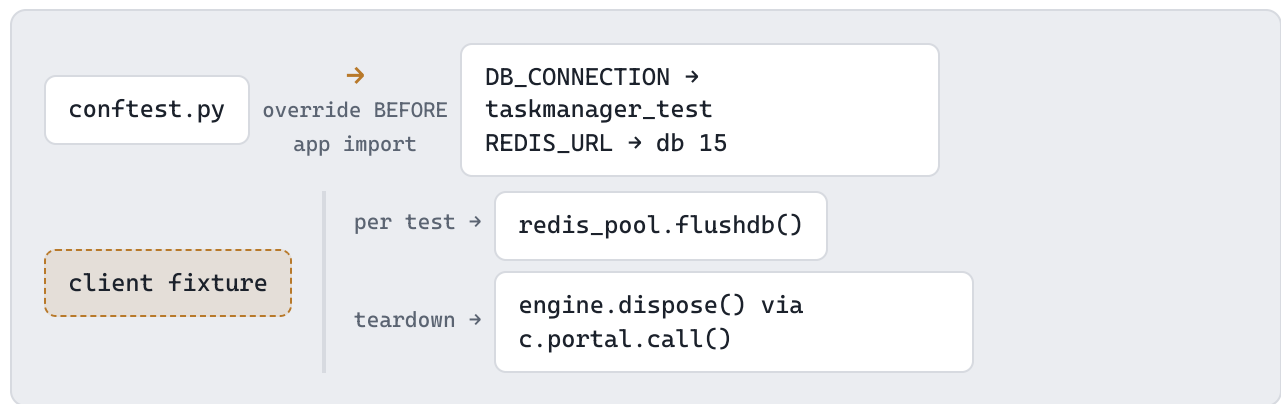
- A properly-sized (32-byte) placeholder JWT secret in CI silenced a legitimate `InsecureKeyLengthWarning` — not suppressed, actually fixed.

CODE — `.GITHUB/WORKFLOWS/TESTS.YML`

```
jobs:
  test:
    env:
      # job-level, not step-level - migrations need these too
      DB_CONNECTION: postgresql:// ... @localhost:5432/taskmanager_test
    services:
      postgres: { image: postgres:16, options: --health-cmd pg_isready }
      redis: { image: redis:7 }
```

32 Test Isolation

INFRASTRUCTURE



Env overrides must happen **before** the app is imported — `Settings()` and `engine` are import-time singletons, too late to redirect afterward. `TestClient` spins up a fresh event loop per test; the module-level `engine` stays bound to whichever loop touched it first, so cross-test `engine.dispose()` has to run through that specific test's own portal, not a bare async call.

DETAILS

- Redis gets the exact same treatment Postgres already had — a separate logical DB (index 15), flushed before every single test, so rate-limit counters and cache entries never leak across unrelated tests.
- A real bug this isolation caught: the whole suite failing only when run *together*, never in isolation — traced to exactly this event-loop binding, not a flaky test.

CODE – CONFTEST.PY

```
os.environ["DB_CONNECTION"] = f"postgresql:// ... @localhost:5432/taskmanager_test
os.environ["REDIS_URL"] = "redis://localhost:6379/15"

@pytest.fixture
def client():
    with TestClient(app) as c:
        c.portal.call(app.state.redis_pool.flushdb) # clean slate every test
        yield c
        c.portal.call(engine.dispose) # same portal, still-alive
```